

Fine-Grained Webpage Fingerprinting Using Only Packet Length Information of Encrypted Traffic

Meng Shen¹, Member, IEEE, Yiting Liu, Liehuang Zhu², Member, IEEE, Xiaojiang Du³, Fellow, IEEE, and Jiankun Hu⁴, Senior Member, IEEE

Abstract—Encrypted web traffic can reveal sensitive information of users, such as their browsing behaviors. Existing studies on encrypted traffic analysis focus on website fingerprinting. We claim that fine-grained *webpage* fingerprinting, which speculates specific webpages on a same website visited by a victim, allows exploiting more user private information, e.g., shopping interests in an online shopping mall. Since webpages from the same website usually have very similar traffic traces that make them indistinguishable, existing solutions may end up with low accuracy. In this paper, we propose *FineWP*, a novel fine-grained webpage fingerprinting method. We make an observation that the length information of packets in bidirectional client-server interactions can be distinctive features for webpage fingerprinting. The extracted features are then fed into traditional machine learning models to train classifiers, which achieve both high accuracy and low training overhead. We collect two real-world traffic datasets and construct closed- and open-world evaluations to verify the effectiveness of *FineWP*. The experimental results demonstrate that *FineWP* is superior to the state-of-the-art methods in terms of accuracy, time complexity and stability.

Index Terms—Webpage fingerprinting, encrypted traffic classification, machine learning, convolutional neural networks.

I. INTRODUCTION

WITH the growth in usage of end-to-end encryption protocols (e.g., SSL/TLS [7], [13]), website fingerprinting has attracted extensive attention in recent years. It is a kind of traffic analysis attack where the adversary attempts to identify the websites visited by victims, by observing specific patterns of encrypted traffic [21]. Considering that a single website

usually consists of multiple webpages, existing studies simply refer to website homepages as their representative webpages. In this paper, we focus on fine-grained *webpage* fingerprinting, which aims to recognize encrypted traffic patterns from specific *webpages* within a given website.

Webpage fingerprinting is capable of exploiting more valuable and private information from encrypted network traffic. For instance, local eavesdroppers are able to identify user browsing behaviors in an online shopping mall and further speculate on users' preferences; they can also recognize specific webpages that a user has visited in a news website, which can be used to analyze her political inclination.

Recent studies have proposed a number of methods for encrypted traffic analysis. Traffic features, such as uplink timing information [11], cumulative packet length [21], and state transition patterns [19], can be extracted from encrypted traffic and then fed to traditional machine learning models (e.g., random forest, k -Nearest Neighbors) to construct classifiers. Deep learning methods such as convolutional neural networks (CNNs) [33] are also applied to improve the accuracy of encrypted traffic analysis. All these schemes, however, aim to construct fingerprints at the granularity of websites or mobile applications, instead of webpages.

It is a challenging task to carry out webpage fingerprinting in an *accurate* and *efficient* way. First, webpages in the same website usually have similar page layouts and adopt the same encryption protocol version (as well as the corresponding parameters), which makes the resulting traffic of different webpages extremely similar, in terms of traffic volume and client-server interaction patterns. As a result, the fingerprints constructed using traffic features exploited in existing literatures, such as cumulative packet length or state transition patterns, are prone to make misclassification (cf. Section VI-B.3). *Second*, webpage fingerprinting requires low computational complexity and quick response time in practical usage [27]. Some intricate methods result in heavy time consumption on either feature extraction [11] or classifier training [33].

In this paper, we propose *FineWP*, a fine-grained webpage fingerprinting approach using only packet length information. In general, packet length and packet time are typically two types of features that can be obtained from encrypted traffic. However, packet time is prone to be affected by network conditions and thus is not as stable as packet length. In addition, existing studies using packet time as features usually employ dynamic time warping to align different time sequences [11], resulting in extremely heavy time overhead in feature extraction. Thus, features based on packet length information are more suitable to build fingerprinting methods.

Manuscript received June 15, 2020; revised October 7, 2020; accepted December 9, 2020. Date of publication December 23, 2020; date of current version February 1, 2021. This work was supported in part by the National Key Research and Development Program of China under Grant 2020YFB1006101; in part by the Beijing Nova Program under Grant Z201100006820006; in part by the NSFC Projects under Grant 61972039 and 61872041; in part by the Beijing Natural Science Foundation under Grant 4192050; in part by the Zhejiang Lab Open Fund under Grant 2020AA3AB04; and in part by the ARC Funds under Grant DP190103660, Grant DP200103207, and Grant LP180100663. The associate editor coordinating the review of this manuscript and approving it for publication was Dr. Tomas Pevny. (Corresponding authors: Meng Shen; Liehuang Zhu.)

Meng Shen is with the School of Cyberspace Security, Beijing Institute of Technology, Beijing 100081, China, and also with the Cyberspace Security Research Center, Peng Cheng Laboratory (PCL), Shenzhen 518066, China (e-mail: shenmeng@bit.edu.cn).

Yiting Liu was with the School of Computer Science, Beijing Institute of Technology, Beijing 100081, China. She is now with the School of Computer Science, Fudan University, Shanghai 200433, China (e-mail: liuyitingbit@foxmail.com).

Liehuang Zhu is with the School of Cyberspace Security, Beijing Institute of Technology, Beijing 100081, China (e-mail: liehuangz@bit.edu.cn).

Xiaojiang Du is with the Department of Computer and Information Sciences, Temple University, Philadelphia, PA 19122 USA (e-mail: dxj@ieee.org).

Jiankun Hu is with the School of Engineering and IT, University of New South Wales, Canberra, ACT 2610, Australia (e-mail: J.Hu@adfa.edu.au).

Digital Object Identifier 10.1109/TIFS.2020.3046876

1556-6021 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: UNIV PUBLICA DE NAVARRA. Downloaded on October 28, 2024 at 07:04:14 UTC from IEEE Xplore. Restrictions apply.

In order to extract discriminative features from encrypted traffic of similar webpages, we take a deep look at the traffic characteristics of bidirectional client-server interaction while loading webpages. A valuable observation is that the entire traffic sequence can be divided into three stages, where the second stage named *uplink-dominant* stage exhibits distinctive patterns across different webpages. Accordingly, we extract three types of features during this stage, i.e., block features, sequence features, and statistical features. These easy-to-obtain features are then incorporated into traditional machine learning models to construct an accurate and efficient classifier.

We conduct extensive experiments using real-world datasets to evaluate the performance of FineWP. In particular, to make a comprehensive comparison with existing methods, we also build CNNs for webpage fingerprinting following the description in the literature [33]. The experimental results show that FineWP is superior to the state-of-the-art methods. In addition, FineWP is 30 $\times$ faster than the CNNs-based approach in terms of training time.

Compared with our previous work [26], we enlarge the experimental dataset, extend the webpage fingerprints by utilizing adequate characteristics, and conduct more performance evaluation in this paper. The additional contributions beyond the original paper are as follows:

- 1) We build two real-world webpage traffic datasets named *JD Dataset* and *YH Dataset*, where JD dataset contains 13,530 webpages with a total of 26,356 sequences and YH dataset contains 283 webpages under 12 labels with a total of 11,023 sequences. Based on these datasets, we conduct extensive experiments to verify the effectiveness of FineWP.
- 2) Based on the three-stage bidirectional client-server interaction, we extract three types of distinctive features only leveraging packet length information and combine them with traditional machine learning models to build FineWP, which enables accurate and computationally efficient webpage fingerprinting.
- 3) We demonstrate the accuracy and efficiency of FineWP through closed-world and open-world experiments. Compared with the state-of-the-art methods, FineWP enables up to 20% improvement on JD dataset and 10% improvement on YH dataset, in terms of classification accuracy. We also observe that FineWP performs comparably to the CNNs-based approach in accuracy, while significantly reducing training time.

The rest of the paper is organized as follows. We first review the related work in Section II. Then, we make an anatomy of webpage fetching interaction in Section III and present the fine-grained webpage fingerprinting method in Section IV. Next, we describe the dataset collection process in Section V and evaluate the performance of FineWP through a comprehensive comparison with existing methods in Section VI. We discuss the limitations of the proposed method as well as future research opportunities in Section VII and conclude this paper in Section VIII.

II. RELATED WORK

Many researchers have proposed a rich number of encrypted traffic analysis methods, such as **website fingerprinting**, **mobile**

TABLE I
SUMMARY OF EXISTING TRAFFIC CLASSIFICATION METHODS

Category	Features
Website Fingerprinting	Cumulative packet length [21] Packet timing information [11] Packet length sequences and time series [34] [33] Distance between packet sequences [37]
Mobile App Fingerprinting	Statistical features of packet length [35] Characteristics of SSL/TLS session [19] [29] [28] Combination of multi-dimensional features [32]
Video Fingerprinting	Total number of bits in a peak [9] Statistical features of packet length and time [8] Round-trip-time [30]

applications fingerprinting, video traffic classification. We make a brief summary of existing methods on encrypted traffic analysis in Table I.

A. Website Fingerprinting Methods

Panchenko *et al.* [21] used cumulated sum of packet size to represent traffic traces, and extracted a fixed number of discriminative features from the traces. However, this method cannot effectively distinguish specific webpage traffic, especially from the same website. Several recent studies applied CNNs to website fingerprinting [24], [25], [33]. The CNNs-based classifiers usually require large-scale training data as well as long training time, thus making this kind of methods complicated and time-consuming.

B. Mobile Application Classification Methods

With the prosperity of smartphones, many recent studies focus on identifying mobile applications via classifying the encrypted traffic generated by these applications [14], [15], [17], [23]. Taylor *et al.* [34] proposed a method called *App-Scanner* that extracts 54 statistic features from incoming flows, outgoing flows and complete flows. Shen *et al.* [28], [29] extended the Markov model [19] by using second-order Markov chains and application attribute bigrams. However, the Markov-based fingerprints are not discriminative enough to accurately identify traffic from webpages of the same website, as evidenced by the experimental results in Section VI.

C. Video Traffic Classification Methods

Dubin *et al.* [9] constructed SVM and nearest neighbor classifiers to match encrypted video traffic with the corresponding video titles. Dong *et al.* [8] proposed a fine-grained classification method for video traffic based on *k*-Nearest Neighbor algorithm. Shen *et al.* [30] proposed a deep learning approach to infer user quality of experience (QoE) from encrypted video traffic. Videos from the same content provider (e.g., YouTube) usually differ greatly in terms of duration and resolution, resulting in more discriminative features of encrypted traffic.

D. The Novelty of This Paper

Existing studies on website fingerprinting usually select a single representative webpage (e.g., homepage) for each website. In this paper, we focus on developing an efficient webpage fingerprinting method, which can differentiate encrypted traffic generated by multiple webpages of the same website.

Webpage fingerprinting is a more challenging task, as the webpages usually have similar or even the same layout, thus resulting similar encrypted traffic patterns. To tackle this problem, we present FineWP that uses only the packet length information. More specifically, it utilizes the features of the bidirectional client-server interactions. The evaluation results demonstrate its superiority in terms of computational complexity and classification accuracy.

III. FINGERPRINTING FOUNDATION: WEBPAGE FETCHING INTERACTIONS

Webpage fingerprinting attempts to associate traffic features with the corresponding webpages from which the traffic is generated. In this section, we first describe the threat model, which determines the way that webpage fingerprinting attacks are conducted by potential adversaries. Then, we briefly introduce typical encryption protocols that are employed by websites to protect traffic contents between clients and servers. Finally, we pay special attention to the client-server interactions when a webpage is loading, which serves as the basis for building effective webpage fingerprints.

A. Threat Model

In this paper, we assume a *local* and *passive* adversary, which is a common assumption in the literature [33]. Specifically, *local* means that the adversary is located in the local network of the victim, and *passive* means that the attacker is able to monitor the victim's network traffic but cannot modify or decrypt the traffic. Potential adversaries can be a campus network administrator or a residential Internet Service Providers (ISP). In this scenario, an adversary obtains the bidirectional (i.e., incoming and outgoing) traffic when a victim is visiting certain webpages, and attempts to infer the corresponding webpages by exploiting distinctive patterns in network traffic.

The adversary who conducts webpage fingerprinting attacks first collects traffic traces by initiating a number of visits from the victim's network, covering the webpages he is interested in identification (i.e., *monitored* webpages). Then, the collected dataset is used by the adversary to extract discriminative features for individual webpages. These features can be used to train a supervised classifier that associates traffic features with the corresponding webpages. Finally, the adversary can collect new traffic traces from the victim's visits and use the classifier to identify the corresponding webpages.

B. Traffic Encryption Protocols

In recent years, we have seen a dramatic growth in the usage of encryption protocols, such as Secure Sockets Layer (SSL) [7] and Transport Layer Security (TLS) [13]. These protocols provide communication security and data integrity over the Internet. They are composed of two layers: the *HandShake* layer and the *Record* layer. The former layer establishes a connection between a client and a server and negotiates a session key, while the latter transfers encrypted data using the generated session key.

The latest TLS v1.3 was released in August 2018. Compared with previous versions, it accelerates encrypted connection and

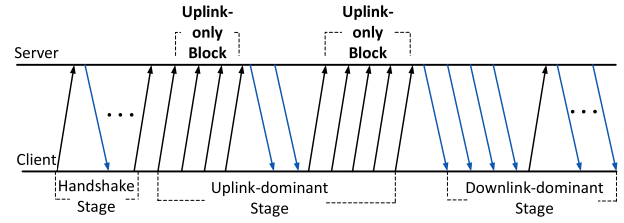


Fig. 1. Typical client-server interactions in webpage loading in terms of uplink and downlink packet sequences.

load time through TLS error start-up and zero round-trip time (0-RTT). It also removes obsolete and unsafe functions from TLS v1.2, such as SHA-1, RC4, and DES. These security and performance enhancements make the encrypted connections more secure and faster than ever.

With SSL/TLS protocols, the content of data transmission can be protected. However, packet number and packet length information of data transmission can still be obtained by potential attackers for further analysis.

C. Webpage Fetching Process

In order to investigate the differences of traffic traces generated from different webpages, we have a careful look at the bidirectional interactions between a pair of client and server during webpage loading.

When a TCP connection is established between a client and a server, it may be reused by the client to request multiple resources (HTML, scripts, and images [6]) from the server so as to reduce connection overhead. At the request sending stage, uplink packets are the mainly transmitted packets. For different webpages, the start time of this stage and the number of uplink packets transmitted at this stage may vary greatly. Fig. 1 illustrates the typical interactions between the uplink and downlink packets¹ during the webpage loading process. Note that the server in this figure *logically* represents all servers that are involved in loading the contents of a webpage.

The interaction can be divided into *three* stages:

1) *Handshake Stage*: This stage transmits the uplink and downlink packets alternately, which mainly aims for the handshake process between the client and the server. For the traffic generated by the same website, the number, length, type and direction of most packets in this stage are extremely similar, as they leverage the same SSL/TLS protocol and adopt the same parameters, e.g., for key negotiation.

2) *Uplink-Dominant Stage*: This stage mainly transmits uplink packets, which is the key stage for webpage fingerprinting. To improve the efficiency of data transmission, a long-lived and high-throughput TCP connection is established by the client to request multiple resources from the server.

At the stage, we can observe one or multiple *uplink-only blocks*, where only uplink packets are transmitted in each block. In practice, we label an uplink-only block only if at least 4 uplink packets are transmitted consecutively, as continuous transmission of less than 4 uplink packets is often caused by unstable network conditions such as packet retransmission.

¹All packets are IP packets and we ignore the specific packet types.

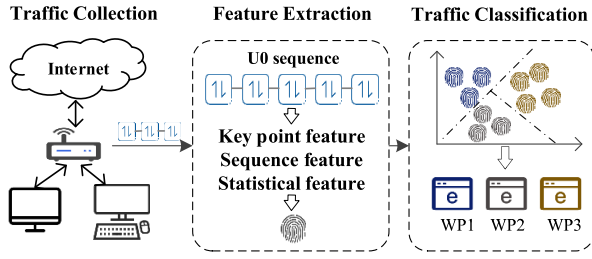


Fig. 2. Overview of FineWP.

Since different webpages have inherent diversities in terms of web element types (e.g., texts, images and other media contents) and element request order, the number of uplink-only blocks as well as the starting and ending positions of each uplink-only block vary among different webpages.

3) *Downlink-Dominant Stage*: This stage alternates the transmission of uplink and downlink packets, while the downlink packets are dominant. That is because the server starts preparing and transmitting the contents required by the client, including HTML header, embedded images, etc.

IV. THE PROPOSED FINEWP

In this section, we present the construction process of the proposed webpage fingerprinting method FineWP. We start with the description of the overall workflow of FineWP. Then, we elaborate on the packet length accumulation process, which facilitates packet sequence abstraction and feature extraction. Finally, we present the construction algorithm of FineWP.

A. Solution Overview

The overall workflow of our solution is depicted in Fig. 2. In the first step, real-world packet sequences are collected by visiting specific webpages.

We observe that features discriminating different pages within the same website exist in the client-server bidirectional interactions, especially in the uplink-dominant stage. The discovery of these features requires an insightful analysis and a novel abstraction of data transmission pattern, which is also the major contribution of this paper.

In the feature extraction process, we accumulate the packet length of original packet sequences and extract three types of features: *block* features, *sequence* features, and *statistical* features. The sequence and block features can fully represent the characteristics of uplink-dominant stage in bidirectional client-server interaction as shown in Fig. 1. In order to further investigate discriminative features, we also employ statistical features with significant contribution to classification accuracy.

Finally, all these features are fed to machine learning algorithms (e.g., random forest, decision tree, and k -nearest neighbors) to build webpage fingerprints. The resulting webpage fingerprints can be applied to classification of unknown traffic traces.

B. Packet Length Accumulation

According to the aforementioned assumption, when a victim visits a certain webpage, the adversary can observe the resulting traffic trace. Inspired by the three stages illustrated

TABLE II
LIST OF NOTATIONS

Notation	Meaning
$\mathcal{D}$	A traffic dataset
$\mathcal{U}$	U0 sequence set of $\mathcal{D}$
P	A packet length sequence in $\mathcal{D}$
U	A U0 sequence in $\mathcal{U}$
K	The number of uplink-only blocks
a	The cumulative packet length in an uplink-only block
s	The order of packet entering an uplink-only block
e	The order of packet ending an uplink-only block
b	Starting position of the sequence feature
d	Ending position of the sequence feature
CS	The contribution score of features

TABLE III
A REAL-WORLD EXAMPLE ILLUSTRATING THREE
TYPES OF PACKET LENGTH SEQUENCES

Type	Packet Length Sequence
Original Sequence	[-66, 66, -54, -571, 60, 1514, 1514, 948, -54, -180, -147, -820, 312, 123, ...]
Processed Sequence	[0, 66, 0, 0, 60, 1514, 1514, 948, 0, 0, 0, 0, 312, 123, ...]
U0 Sequence	[0, 66, 66, 66, 126, 1640, 3154, 4102, 4102, 4102, 4102, 4414, 4537, ...]

in Fig. 1, we further investigate the diversity of traffic traces in terms of packet length in the uplink-dominant stage.

We start with representing the original trace of packet lengths by $P = (p_1, \dots, p_N)$, where $p_i > 0$ indicates an incoming packet and $p_i < 0$ an outgoing packet [21]. The notations used in this paper are summarized in Table II.

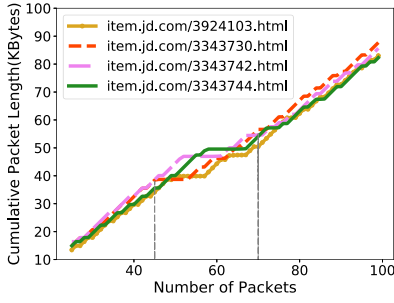
For ease of identifying the appearance of uplink-only blocks in P , we set the length of all outgoing packets as 0 and keep the length of all incoming packets as is. More precisely, $p_i > 0$ for all incoming packets and $p_i = 0$ for all outgoing packets.

Next, we accumulate the packet lengths to obtain an uplink-zero sequence (*U0 sequence* for short), which can be represented by $U = (u_1, \dots, u_N)$, where u_i is obtained in Eq. (1).

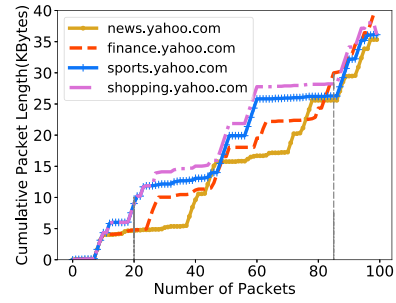
$$u_i = \begin{cases} p_i, & \text{if } i = 1 \\ u_{i-1} + p_i, & \text{if } 1 < i \leq N \end{cases} \quad (1)$$

Based on the analysis of webpage traffic interaction, we envision step-growth of the U0 sequence in the uplink-dominant stage, where the cumulative packet length would remain the same for each uplink-only block. Since different webpage traffic have discriminations in the uplink-dominant stage, which has been explained in Section III, U0 sequence can extract such differences.

Table III presents an example illustrating the original sequence, processed sequence (i.e., by setting $p_i = 0$ for all outgoing packets), and U0 sequence.

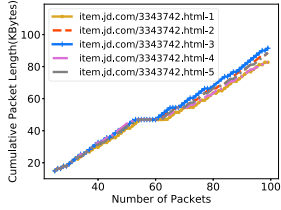


(a) U0 sequences of 5 different webpages in JD dataset

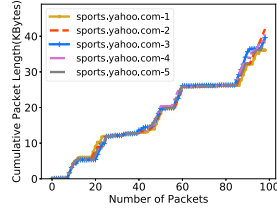


(b) U0 sequences of 4 different webpages in YH dataset

Fig. 3. U0 sequence samples for different webpages in JD and YH datasets.



(a) U0 sequences of a webpage in JD dataset



(b) U0 sequences of a webpage in YH dataset

Fig. 4. U0 sequences samples for the same webpage in JD and YH datasets.

To have an intuitive sense of how U0 sequences extract differences among webpages within the same website, we randomly select 4 samples from JD and YH datasets respectively and visualize their U0 sequences in Fig. 3. The 4 webpages exhibit the same type of products with different brands and 4 webpages from YH dataset with different labels. The details of the datasets will be described in Section V. We also exhibit the U0 sequences generated by 5 different visits of the same webpages in Fig. 4.

We can see that in the uplink-dominant stage (i.e., between the 45th and 70th packets in JD dataset and between the 20th and 85th packets in YH dataset), the traffic sequences from different webpages become apparently distinguishable, while the U0 sequences are almost consistent over consecutive downloads of the same webpage.

A transmission-level implication of these differences lies in that different webpages have inherent diversities in terms of web element types (e.g., texts, images and other media contents) and element request order. This result strongly confirms that U0 sequence could express the client-server interaction and contribute to distinguishable webpage fingerprints.

As a result, finding the cumulative packet length as well as the starting and ending positions of packets corresponding to the uplink-dominant stage based on U0 sequence should be an effective way to construct discriminative fingerprints for different webpages.

C. Sequence Abstraction and Feature Extraction

Our key observation above clearly indicates that U0 sequence can express webpage differences in terms of packet-level interactions. To characterize the common pattern of concrete U0 sequences, we propose a U0 sequence abstraction.

The U0 sequence abstraction in Fig. 5 captures the common trend in U0 sequences with one or multiple uplink-only blocks. Formally, it has the format of $\langle K, B_1, \dots, B_K \rangle$, where K is the total amount of uplink-only blocks and the i -th block $B_i = (s_i, e_i, a_i)$. As illustrated in Fig. 5, s_i and e_i indicate the orders of the packets starting and ending the i -th block, and a_i represents the corresponding cumulative packet length. Such an abstraction can flexibly define different U0 sequences by setting appropriate parameters.

The U0 sequence abstraction enables us to extract critical features for building efficient webpage fingerprints, including *block features*, *sequence features*, and *statistical features*.

1) *Block Features*: They are the representative features for uplink-only blocks in U0 sequences. More precisely, we refer to $B_i (i \in [1, K])$ as the block features. These features are the most crucial features among all types of features, as they concisely describe the number of uplink-only blocks and their appearance positions, as well as the transition of cumulative packet length among consecutive uplink-only blocks.

2) *Sequence Features*: This type of features refer to the packet length subsequence in the duration of the uplink-dominant stage. They are crucial to build webpage fingerprints, as packet sequences of different webpages in the uplink-dominant stage are distinguishable.

For a specific U0 sequence with K uplink-only blocks as shown in Fig. 5, the starting and ending positions of the uplink-dominant stage can be easily defined by s_1 and e_K . However, these positions of U0 sequences can be different for multiple visits of the same webpage and vary even more significantly for different webpages. In order to make sure that the sequence feature has the same dimension for different U0 sequences, we should determine the *unified* starting and ending positions of the uplink-dominant stage for all sequences in a dataset.

Since there is usually a high volume of traffic sequences in a dataset, taking an intersection or union of s_1 and e_K across all sequences will make the resulting range too small or too large, thus weakening the effectiveness of the features. Therefore, we calculate the mean values of s_1 and e_K of all samples, respectively, as the range of the uplink-dominant stage $[b, d]$, which is defined in Eq.(2),

$$b = \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} s_1^i, \quad d = \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} e_K^i \quad (2)$$

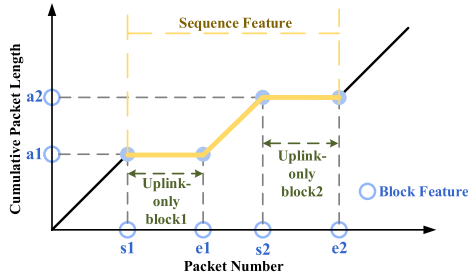


Fig. 5. The sketches of uplink-only block, block feature and sequence feature.

Thus, the sequence features can be defined as $SF = [u_b, u_{b+1}, \dots, u_d]$.

3) *Statistical Features*: They are extracted from the original packet length sequence. For each sequence $P \in \mathcal{P}$, we divide it into three series: uplink packet length series, downlink packet length series, and complete packet length series (i.e., the combination of uplink and downlink packets). Motivated by the literature [34], we calculate the following statistical features for each series: minimum, maximum, mean, median absolute deviation, standard deviation, variance, skew, kurtosis, percentiles (from 10% to 90%), and the number of packets.

We utilize the Random Forest (RF) classifier implemented in scikit-learn to get the Gini impurity index, which is computed during estimator building to evaluate the contribution of statistical features [20]. The contribution of each individual feature is quantified by its contribution score (CS). In practice, to reduce the feature dimension while ensuring the accuracy, we only reserve the statistical features with a CS over a certain threshold. We will justify this strategy in Section VI-B.3.

4) *Remarks*: We provide an insightful analysis why these features discriminate different webpages within the same website. According to the webpage fetching process in Fig. 1, the block features can roughly reflect the characteristics of element requests in a visit to a webpage, such as the number of element requests, and the packet subsequence and cumulative packet length for each request. The sequence features can extract the starting and ending positions of the uplink-dominant stage for multiple visits to the same webpage. The statistical features consider the webpage diversity (e.g., contents and layout) and provide valuable summaries of the packet-level information throughout the webpage fetching process.

D. Construction of *FineWP*

With the features described above, we now detail the construction of *FineWP*. For ease of illustration, let $\mathcal{D}$ denote a dataset (e.g., JD) consisting of packet traces from multiple webpages. According to the aforementioned description of U0 sequences, we can obtain the U0 sequence set $\mathcal{U} = \{U_1, \dots, U_{|\mathcal{D}|}\}$, where the i -th sequence is denoted by $U_i = (u_1^i, \dots, u_N^i)$.

The construction of *FineWP* is depicted in Algorithm 1. We first preprocess the original traffic (as shown in Section V-B) and utilize U0 sequence $U = (u_1, \dots, u_N)$ to describe the webpage loading process (lines 1-2). Next, to obtain all features, we define $A = \{u : c\}$, where u is an element of U and c represents the count of u with the same

Algorithm 1 Construction of *FineWP*

Input: A packet length sequence $P = (p_1, \dots, p_N)$ resulting from a webpage loading process

Output: Webpage fingerprinting model *FineWP*

- 1: Set the uplink packet length to 0
- 2: Accumulate the packet length to obtain U0 sequence $U = (u_1, \dots, u_N)$
- 3: Initialize $A = \{u : c\}$, $S = \{u : s\}$ and $E = \{u : e\}$
- 4: **for** $u_i \in U$ **do**
- 5: **if** $u_i = u_{i+1}$ **then**
- 6: **if** A find u_i **then**
- 7: $A[u_i] + 1$
- 8: **else**
- 9: $S[u_i] \leftarrow i$
- 10: **if** A find u_i and $u_i! = u_{i+1}$ **then**
- 11: $E[u_i] \leftarrow i$
- 12: Sort A , S and E according to c and get block feature $B = \{(s_1, e_1, u_1), \dots, (s_K, e_K, u_K)\}$
- 13: Calculate b and d according to Eq. (2)
- 14: Obtain the sequence feature $SF = \{u_b, u_{b+1}, \dots, u_d\}$
- 15: Calculate statistical features ST of P
- 16: Reserve elements of ST with a CS larger than a threshold
- 17: Obtain the final feature set $feature = \{B, SF, ST\}$
- 18: $kNN, RF, DTree \leftarrow feature$
- 19: **return** Webpage fingerprinting model

value. Note that we record the data into A only when $c \geq 4$, because it meets the requirement of uplink-only block. Define $S = \{u : s\}$, where s represents the order of the packet entering the uplink-only block. Define $E = \{u : e\}$, where e represents the order of the packet ending the uplink-only block (line 3).

More specifically, we extract all distinctive features by going through U and recording the data in A , S and E (lines 4-11). With regard to the sequence feature, we record s_1 and e_K for each U and leverage the mean values of them as the unified starting and ending positions (lines 13-14). The final starting position b and ending position d can be obtained according to Eq. (2). Then, the sequence feature can be denoted as $SF = (u_b, u_{b+1}, \dots, u_d)$, where $u \in U$ and $b < d$.

Finally, we utilize random forest to measure the contribution of statistical features. Conveniently, random forests implemented in scikit-learn have already collected information about the feature contribution for us. After fitting Random Forest Classifier, we can get the CS according to feature contributions. We add statistical features with CS higher than a certain threshold to the feature set (lines 15-16).

We use three typical supervised machine learning methods as our classifiers: k-Nearest Neighbor (kNN) [35], Random Forest (RF) [20] and Decision Tree (DTree) [22]. We feed these webpage features into the classifiers to construct webpage fingerprints (lines 18-19). The optimal choice of machine learning classifiers will be discussed in Section VI-B.

V. DATASET COLLECTION

In this section, we describe the dataset that is used for evaluating the performance of *FineWP*.

TABLE IV
DATASETS

Datasets	# webpages	# labels	# sequences	Size
JD	330 (Monitored) 13200 (Unmonitored)	N/A	13,156 (Monitored) 13,200 (Unmonitored)	26.4 GB
YH	283	12	11,023	19.9 GB

A. Data Description

Since there is no publicly-available webpage traffic dataset, we collect real-world webpage traffic datasets using 4 *Ali* cloud ECS servers [1] located in different regions in China. The cloud servers are used as clients to simulate users' browsing behaviors. Two cloud servers are equipped with Windows 8 and the rest are equipped with Windows 10. To emulate the diversity of browsers, we employ Chrome, Mozilla Firefox, Internet Explorer, and 360 Browser in data collection. We consider the resulting traffic as a whole to make the trained classifiers applicable to variations of operating systems and browsers.

All traffic traces are captured using Wireshark, i.e., we employ tshark scripts to automatically capture the traffic generated by visiting a specific webpage for 60 seconds. This duration can guarantee that all elements in a webpage are successfully loaded by the browser in general cases. The scripts empty the caches each time visiting a certain webpage and close the browser when each visit is completed. The data collection is conducted within a period of two months.

JD represents one of the largest online shopping malls, Jingdong [2], in China. Each commodity is exhibited on a unique webpage, and the templates of different webpages are very similar. To construct the dataset for *closed-world* evaluation, we randomly select 330 webpages on JD as the monitored webpages, each of which has 40 individual visits. To construct the dataset for *open-world* evaluation, we randomly select 13,200 webpages as the unmonitored webpages, which are different from the monitored ones.

Our choice follows the common rules to construct open-world dataset in the literature [21], [33]. In a more realistic open-world scenario, our goal is to investigate to what extent a fingerprinting method can recognize a small number of monitored webpages from a much larger amount of unknown, randomly-accessed webpages that the classifier has never seen before. In other words, the classifier can train on the monitored webpages but cannot train on the unmonitored webpages.

YH represents *Yahoo!* [5], which is one of the most popular websites in the world [4]. We randomly select a total number of 300 webpages from 12 categories (also referred to as *labels*²), and each webpage has 40 individual visits.

We select these representative websites because: 1) they have a large number of daily visitors, and 2) the browsing behaviors on online shopping and portal sites provide an opportunity for an adversary to infer personal preferences.

²The 12 labels on Yahoo! include Entertainment, Finance, Lifestyle, Mobile Apps, News, Security, Shopping, Small Business, Sports, Weather, Developer Network and Answers.

B. Traffic Preprocessing

Each packet contains the following information: source and destination IP addresses, source and destination port numbers, communication protocol, timestamp, packet length and packet flags (e.g., SYN, ACK).

In order to select the packets generated from the two websites of interest, we resort to the server name indication (SNI) field [3] in a TLS Client Hello message.³ That is, if “jd.com” or “yahoo.com” or “yimg.com” (indicating image loading of Yahoo webpages) exists in SNI, we record the corresponding IP addresses as the server addresses (i.e., source IP addresses in downlink packets and destination IP addresses in uplink packets). Since a website usually involves multiple servers with different IP addresses, the collection of server IP addresses enables us to reserve the desirable packets.

It should be noted that the packets corresponding to advertisements on webpages are discarded, as the advertisements on the same webpage usually dynamically change for different visits. In addition, we remove from our datasets those webpages that fail to load due to poor network conditions. This is conducted by comparing their screenshots with a blank page, an access denied page, or a timeout-error page.

With the aforementioned preprocessing, we obtain an original packet sequence for each webpage visit, which composes of a series of IP packets sorted by their timestamp marked with Wireshark. The refined datasets are summarized in Table IV. The JD dataset contains a total of 13,156 sequences from 330 monitored webpages and 13,200 sequences from 13,200 unmonitored webpages. The YH dataset consists of a total of 11,023 sequences from 283 webpages with 12 labels, and each label has 23 different webpages on average.

VI. PERFORMANCE EVALUATION

In this section, we are dedicated to evaluate the performance of FineWP. We first introduce the experiment setting and parameter setting. After that, we thoroughly compare FineWP with the state-of-the-art methods in terms of classification accuracy and time complexity in the closed-world and open-world settings.

A. Experiment Setting

1) *Methods to Compare*: In order to have a comprehensive understanding of the performance of FineWP, we leverage 4 existing methods for comparison.

- **WFCNN** is a CNNs-based classifier that follows the construction details in [33]. The classifier takes as the input the sequences of tuples (*timestamp*, *packet_length*), where the sign of packet length indicates the direction of the packet: positive means downlink packets and, negative, uplink packets. We conduct an extensive search through hyperparameter space to make the classifier achieve its best performance. The selected hyperparameters for two datasets are shown in Table V.
- **CUMUL** is a website fingerprinting approach based on the feature of cumulative packet lengths [21]. The uplink

³SNI is an important extension for SSL/TLS protocol that allows virtual site hosting, e.g., multiple sites can be hosted on a single IP address and each site has its own TLS certificate.

TABLE V
HYPERPARAMETERS SELECTION FOR WFCNN

Hyperparameters	Search Range	Values on JD	Values on YH
Input Dimension	[500...7000]	4000	2000
Optimizer	[Adam, Adamax, RMSProp, SGD]	Adamax	Adamax
Learning Rate	[0.001...0.01]	0.002	0.002
Training Epochs	[10...100]	50	30
Batch Size	[16...256]	32	64
Activation Functions	[Tanh, ReLU, ELU]	ELU, ReLU	ELU, ReLU
# of Block1 [Conv1]	[8...64]	32	32
# of Block2 [Conv2]	[32...128]	64	64
Pooling Layers	[Average, Max]	Max	Max
# of FC Layers	[1...4]	2	2
Hidden units	[64...2048]	256	128
Dropout [Pooling, FC1, FC2]	[0.1...0.8]	[0.1, 0.7, 0.5]	[0.1, 0.7, 0.5]

packet length is set to negative and the downlink packet length is set to positive. Then, the accumulative packet length sequence is used to represent traffic traces. The resulting feature set contains the first 100 cumulative packet lengths as well as the total number of uplink, downlink and complete packets.

- *WPF* is our previous work [26], which leverages a 2-dimensional characteristics. We set the uplink packet length to 0 and accumulate the packet length sequences. The feature set is the cumulative packet length and the count of packets in the uplink-dominant stage.
- *AppS* is a random forest classifier to identify smartphone applications from encrypted traffic [34]. It uses 54 statistics features of uplink flow, downlink flow, and complete flow as the feature set. This method is also widely used in website fingerprinting.

2) *Cross-Validation*: To validate webpage fingerprints created by different methods, we divide each dataset into the training set and the testing set. To mitigate the impact of dataset partitioning on classification performance, we apply 10-fold cross-validation. We consider a dataset as a whole and divide it into 10 equal parts. Then, we use one part as the testing set and the rest as the training set. After 10 rounds of tests, we record the average of results.

To avoid learning on artifacts specific for individual users, we also adopt another cross-validation strategy: the traffic traces collected on 3 cloud servers are used for training while those collected on the 4th server are used for testing. The evaluation results will be described in Section VI-E.

3) *Performance Metrics*: The fundamental goal of webpage fingerprinting is to be accurate, i.e., identifying more encrypted traffic traces correctly and avoid misclassification.

In the closed-world setting, we consider *accuracy* as the performance metric for a multi-class classifier, which is defined as the total number of correct classifications (*TC*) divided by the total number of classifications (*Sum*), i.e., $Acc = TC/Sum$.

In the open-world setting, the classification is usually treated as a *binary* classification problem. Thus, we utilize *precision*

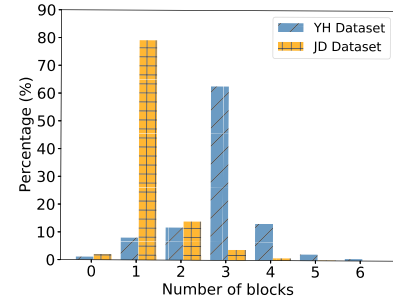


Fig. 6. Number of uplink-only blocks for YH and JD datasets.

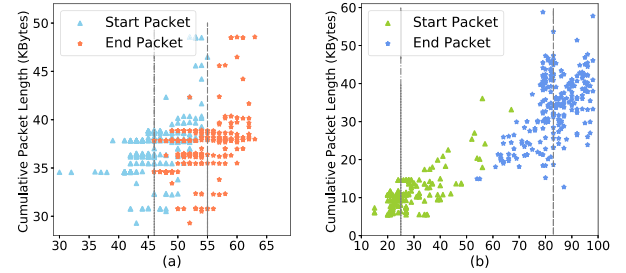


Fig. 7. The range of sequence features for JD dataset (a) and YH dataset (b).

and *recall* as performance metrics of the classifiers [33]. These metrics are defined on 4 basic concepts that are commonly used in the literature [16], namely True Positive (TP), False Positive (FP), True Negative (TN) and False Negative (FN). Then, precision and recall can be calculated $Pre = TP/(TP + FP)$ and $Rec = TP/(TP + FN)$, respectively.

B. Parameter Optimization of FineWP

The selection of parameters directly affects the performance of FineWP. In this section, we introduce the selection strategies of feature parameters and classifies.

1) *The Number of Uplink-Only Blocks*: In order to extract block features, we need to determine the number of uplink-only blocks K for both JD and YH datasets. We only consider the packets between 20 and 100, since the first 20 packets usually belong to the handshake stage. In practice, a subsequence with more than 4 continuous uplink packets is regarded as an uplink-only block. Fig. 6 presents the proportion of the number of blocks in JD and YH datasets. According to the results, K is set as 1 and 3 for JD and YH datasets, respectively.

2) *The Range of Sequence Features*: In order to make sure that the sequence feature has the same dimension for different U0 sequences, we should determine the *unified* starting and ending positions of the uplink-dominant stage for all sequences in each dataset. Fig. 7 exhibits the start and ending positions of the uplink-dominant stage for each U0 sequence in JD and YH datasets. We can find that the range of the uplink-dominant stage varies a lot among different sequences.

In our experiment, we apply mean value calculation to extract the range of sequence features for each dataset. For JD dataset, we intercept the 46th to 55th packets of U0 sequence as the sequence features. While for YH dataset, we extract the subsequence between 25 and 83 as the sequence features.

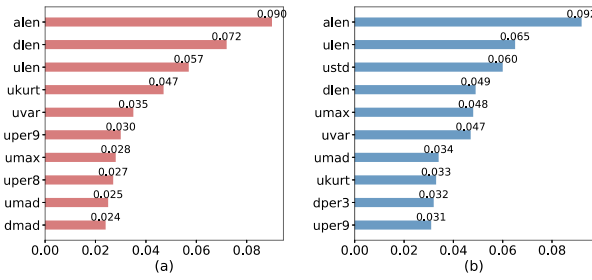


Fig. 8. Ranked contributions of individual statistical features on JD dataset (a) and YH dataset (b).

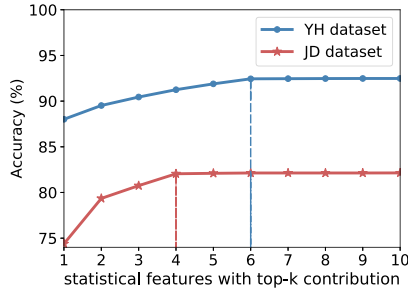


Fig. 9. Effect of statistical features on classification accuracy.

3) *Statistical Features*: For each traffic sequence, three packet series are considered: uplink-only packet series (*u-*), downlink-only packet series (*d-*) and complete packet series (*a-*). Motivated by [34], we calculate 18 statistical features (a total of 54 features) for each series: minimum (*min*), maximum (*max*), mean (*mean*), median absolute deviation (*mad*), standard deviation (*std*), variance (*var*), skew (*skew*), kurtosis (*kurt*), percentiles (from 10% to 90%) (*per1-per9*) and the number of elements (*len*) in the series. The statistical features can be represented by the combination of series type and feature type, e.g., *alen* represents the number of elements in the complete packet series. Then, we calculate the feature contributions based on the method introduced in Section IV-C.

Fig. 8 exhibits the top 10 features that contribute most to the classification for JD and YH datasets. We can see that the scores of statistical features are relatively low (i.e., less than 0.1), which indicates that these features have limited contribution to classification. It is because the page layouts of webpages on the same website is similar, resulting in very similar traffic generated from the client-server interactions.

According to the contribution ranking, we add these statistical features into the feature set successively and obtain the classification accuracy, as shown in Fig. 9. We can see that the features with a *CS* less than 0.04 contribute trivially to accuracy improvement. Thus, the threshold in Algorithm 1 is set as 0.04. For JD dataset, 4 statistical features meet the requirement, including *alen*, *dlen*, *ulen* and *ukurt*. And for YH dataset, 6 statistical features are involved, including *alen*, *ulen*, *ustd*, *dlen*, *umax* and *uvar*.

4) *Machine Learning Classifier Selection*: As stated in Fig. 2, FineWP is built by incorporating traffic features into traditional machine learning classifier. RF, DTree and k-NN are chosen as candidate classifiers, because they are particularly suitable for predicting classes (i.e., webpages in

TABLE VI
HYPER-PARAMETERS OF ML ALGORITHMS

Classifiers	Parameters
<i>k</i> -NN	<i>k</i> =1 (JD), <i>k</i> =5 (YH)
RF	<i>max_features</i> =None, <i>n_estimators</i> =10, <i>min_samples_leaf</i> =1, <i>max_depth</i> =None
DTree	<i>max_depth</i> =None, <i>criterion</i> ='gini', <i>min_samples_split</i> =2

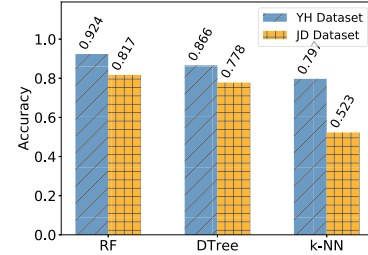


Fig. 10. Classification results with different ML classifiers.

our case) when trained with the preselected features described above.

Since the webpage fingerprints is not high-dimensional, we apply the default hyper-parameters of RF and DTree in scikit-learn for classification. As the effect of *k*-NN is sensitive to the value of *k*, we adjust *k* to get its best performance. The details of the hyper-parameters are shown in Table VI.

Fig. 10 exhibits the classification accuracy with three classifiers. RF performs the best on both datasets, slightly better than DTree. That is because RF consists of multiple decision trees, and the final prediction results are voted by all the decision trees. *k*-NN has the worst performance among the three classifiers. The reason lies in that *k*-NN has pass-through effect, that is, if the abnormal traffic in the training dataset is just next to the traffic sequences we need to classify, it will directly lead to inaccurate prediction data. According to the experiment results, we select RF as our classifier.

C. Closed-World Evaluation

In this section, we evaluate the performance of FineWP in the closed-world setting with JD and YH datasets. In this scenario, we assume that only the monitored webpages can be visited by users, and webpage fingerprinting thus turns into a multi-class classification problem. Although this scenario is not realistic, it is suitable for comparing the classification performance of different classifiers [21].

In order to present a comprehensive understanding of the contribution of FineWP, we leverage the other four methods for comparison: WPCNN, CUMUL [21], WPF [26] and AppS [34]. Note that we apply RF as the classifier of CUMUL instead of SVM as described in their work [21], because CUMUL using RF performs better on our datasets.

1) *Classification Accuracy*: Fig. 11 presents the performance of different methods on JD and YH datasets, respectively. From the results, we can see that FineWP performs much better than the other methods on JD dataset. While on YH dataset, the accuracy of both FineWP and WFCNN is over 90%, higher than the rest methods.

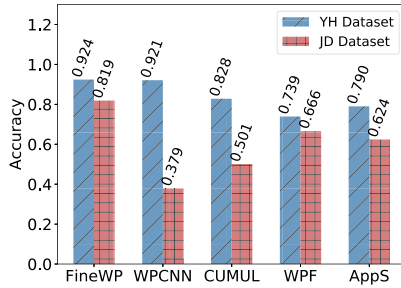


Fig. 11. Closed-world: Accuracy of different methods.

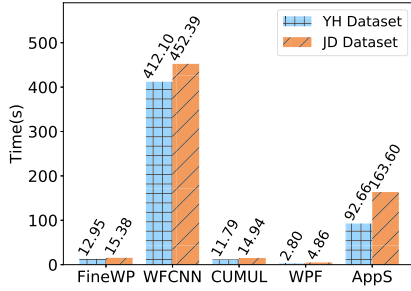


Fig. 12. Run time of different methods in training stage.

We further analyze the reasons for the performance disparity of WFCNN on two datasets. For YH dataset, we apply a label-level classification. There are 12 labels, with an average of 20 pages per label and 40 visits per page. Therefore, each label has around 800 instances. However, in JD dataset, each webpage only has 40 instances. In order to mitigate the problem of data insufficiency in training stage, we randomly select 20 webpages in JD dataset and visit each page 800 times. Following the method presented in Section V-B, we process the data and retrained the WFCNN classifier. As a result, its classification accuracy increases to 71.3%, which is still around 10% lower than that of FineWP.

In terms of CUMUL, it merely achieves an accuracy of 50.1% on JD dataset, but performs well on YH dataset with an accuracy of 82.8%. Now, we have a deep look at the reason why CUMUL performs differently on two datasets. The feature set of CUMUL mainly contains the first 100 cumulative packet length. According to the results of extracting sequence features in Section VI-B, the traffic sequences between 25 and 83 packets are discriminative on YH dataset, while on JD dataset, distinguishable packet sequences are from 46 to 55. The feature set of CUMUL has a high coincidence with the sequence feature of YH dataset, which is the main reason of its effectiveness on YH dataset. This result also confirms the validity of sequence features proposed in this paper.

In terms of WPF, the inadequate features directly lead to its ineffectiveness. As for AppS, the reason for its poor performance is that the contributions of statistical features on both datasets are relatively low (less than 0.1), which has been confirmed in Section VI-B).

The closed-world experimental results confirm the effectiveness of FineWP in fine-grained webpage fingerprinting.

2) *Time Complexity*: Fig. 12 and Fig. 13 exhibit the training time and the validation time of the five methods on JD and YH datasets, respectively.

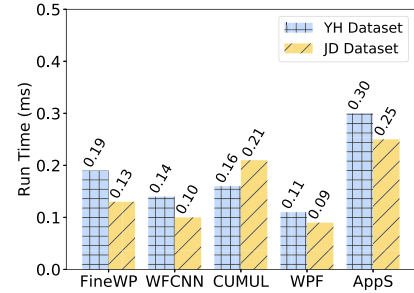


Fig. 13. Run time of different methods in validation stage.

TABLE VII
THE IMPACT OF INSTANCE SCALE ON ACCURACY

Dataset	Methods	Number of Instances			
		15	25	35	MAX
JD Dataset	FineWP	0.76	0.80	0.81	0.82
	WFCNN	0.15	0.21	0.35	0.38
	CUMUL	0.40	0.45	0.49	0.50
	WPF	0.52	0.59	0.62	0.66
	AppS	0.57	0.58	0.60	0.62
YH Dataset	FineWP	0.86	0.89	0.91	0.92
	WFCNN	0.76	0.81	0.89	0.92
	CUMUL	0.75	0.77	0.80	0.82
	WPF	0.68	0.70	0.72	0.74
	AppS	0.72	0.74	0.78	0.79

The run time of these methods in the training stage includes the total time of feature extraction and training. We can see that the run time of WFCNN is the longest among the five methods. That is because during the training stage of CNN, a large number of computations are required through the convolution layer, pooling layer, etc, thus leading to a high time complexity. On the contrary, the feature dimensions of FineWP, CUMUL and WPF are low and the calculations of these features are simple. Therefore, compared with WFCNN, these methods cost much less time in the training stage.

As for the validation stage, the run time in the y-axis is averaged by the number of sequences. There is slightly difference in the running time of the five methods. Note that we take the final prediction process as the validation time of WFCNN. Although this stage does not take long time, WFCNN is still the most time-consuming method considering the total run time spent on training and validation.

To sum up, with the highest accuracy, the time complexity of training FineWP is also relatively low. With a joint consideration of accuracy and time complexity on both datasets, we can conclude that FineWP is superior to its counterparts.

3) *Learning Capability*: We investigate the learning capability of these methods with varying training dataset scales, including the number of instances of each webpage and the number of webpages in the training datasets.

a) *Performance with a varying number of instances*:

As mentioned the previous section, an instance of a webpage represents a packet sequence generated by visiting this webpage, e.g., 30 instances of a webpage can be collected

TABLE VIII
THE IMPACT OF INSTANCE SCALE ON RUN TIME

Dataset	Methods	Number of Instances			
		15	25	35	MAX
JD	FineWP	10.76s	12.18s	14.01s	15.38s
	WFCNN	376.11s	405.80s	431.76s	452.39s
	CUMUL	9.06s	10.77s	12.98s	14.97s
	WPF	2.14s	3.29s	4.01s	4.86s
	AppS	116.28s	129.79s	152.50s	163.60s
YH	FineWP	9.07s	10.94s	12.03s	12.95s
	WFCNN	361.51s	385.79s	400.22s	412.10s
	CUMUL	7.00s	8.25s	10.69s	11.79s
	WPF	1.20s	1.74s	2.59s	2.80s
	AppS	62.13s	78.88s	89.90s	92.66s

if this webpage is visited 30 times successfully. By varying the number of instances for training, we can investigate the data requirements of each method to achieve its desirable performance.

There are 40 instances of each webpage in the original dataset of JD and YH, but the number of instances in each webpage is different after filtering out the abnormally loaded webpages through data processing. Therefore, we set the number of instances of each webpage in turn to be 15, 25, 35 and Max (representing all instances of each webpage) and calculate the accuracy and run time of different methods on JD and YH datasets, respectively. Note that the run time refers to the total time spent by the classifiers on the training stage.

Table VII presents the impact of the number of instances on the classification accuracy. The experimental results show that FineWP is superior to the state-of-the-art methods no matter how many instances there are. When the number of instances reaches 25, the accuracy of FineWP can reach more than 80%. As for WFCNN, its performance is greatly affected by data volume. With the number of instances decreases, the accuracy of WFCNN drops rapidly. The results demonstrate that FineWP has the ability to effectively classify webpages with limited training data.

Table VIII shows the impact of the number of instances on the run time. Note that the run time refers to the total time spent by the classifier in the training stage. From the results we can see that, compared with WFCNN, FineWP can speed up the training process by around 30 $\times$. More importantly, the training time of FineWP does not grow significantly with the increase of the number of instances.

b) Performance with a varying number of webpages:

By analyzing the impact of the number of webpages on the classifiers, we could identify whether a classifier can cope with the condition that the number of classification categories greatly increase. For YH dataset, the classification granularity is label, i.e., only 12 types of labels are involved for classification. Thus, the YH dataset is not very suitable for this evaluation purpose and we only analyze the performance of different classifiers on JD dataset.

Fig. 14 and Table IX respectively show the impact of webpage scale on accuracy and run time of the five classifiers.

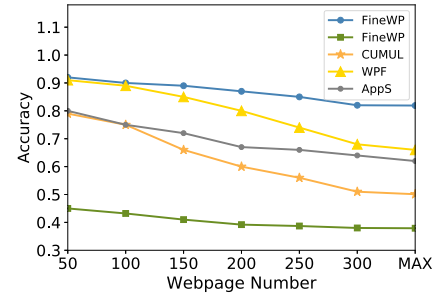


Fig. 14. The impact of webpage scale on accuracy.

TABLE IX
THE IMPACT OF WEBPAGE SCALE ON RUN TIME

Methods	Number of Webpages						
	50	100	150	200	250	300	MAX
FineWP	9.8s	10.7s	11.5s	12.4s	12.9s	14.1s	15.4s
WFCNN	328.7s	351.2s	360.8s	379.0s	411.2s	439.7s	452.4s
CUMUL	6.9s	8.1s	9.8s	11.0s	12.2s	14.1s	15.0s
WPF	1.3s	1.8s	2.0s	2.9s	3.6s	4.2s	4.9s
AppS	101.0s	119.9s	123.4s	130.6s	144.7s	155.5s	163.6s

The run time represents the total time spent by the classifiers in the training stage. It can be seen from the experimental results that with the increase of the number of webpages, the accuracy of the five methods gradually reduces while the time consumption increases, where WFCNN shows the lowest accuracy and the longest time consumption. Compared with the other three methods except WFCNN, the accuracy of FineWP declines slowest and the increase of run time is relatively slow. This result strongly proves that FineWP has the ability to deal with the situation where the number of webpages to be classified greatly increases.

To sum up, FineWP does not need much training data and has low time cost, so it is effective and stable enough to be leveraged in webpage fingerprinting.

D. Open-World Evaluation

In order to compare the performance of classifiers in a more realistic scenario, we leverage the complete JD dataset, i.e., traces from 330 monitored pages and 13,200 unmonitored pages, to evaluate all the methods.

In the open-world setting, the foreground data is referred to as a set of monitored webpage traces, and the background data is a set of unmonitored webpage traces. The attacker attempts to identify whether the webpage visited by a victim is monitored or not. Therefore, in the open-world setting, traffic analysis is a *binary* classification problem. Following the metrics commonly used in the literature [33], we leverage *precision* and *recall* to evaluate the performance of classifiers, as they can avoid the base rate fallacy.

Figs. 15 and 16 illustrate the classification results of the five methods with varying numbers of background webpages. It can be seen from the figure that, except WFCNN, the precision and recall of all classifiers decrease with the increase

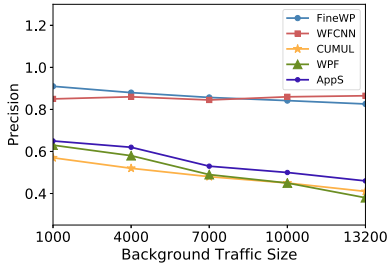


Fig. 15. Open-world: Precision of different methods with varying background traffic size.

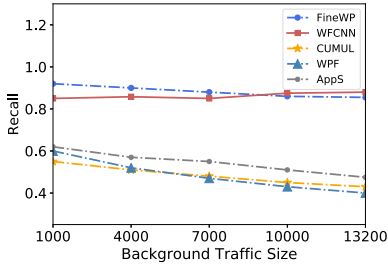


Fig. 16. Open-world: Recall of different methods with varying background traffic size.

TABLE X
OPEN-WORLD: TIME COST OF DIFFERENT METHODS

Method	Number of background pages				
	1000	4000	7000	10000	13200
FineWP	16.6s	17.8s	20.7s	22.2s	26.5s
WFCNN	463.3s	545.5s	669.0s	722.4s	871.2s
CUMUL	16.1s	18.2s	21.0s	24.9s	29.3s
WPF	5.0s	6.7s	7.3s	8.8s	10.3s
AppS	165.9s	185.8s	219.0s	242.2s	288.9s

of the size of background webpages, and FineWP has the best classification effect compared with the other three methods. Moreover, with the increase of background webpages, precision and recall of FineWP decrease the least.

In terms of WFCNN, it classifies traffic based on CNNs, and thus its performance greatly relies on the size of data, i.e., the larger the size of data, the more accurate the CNNs-based classifier. With the increase of background traffic, WFCNN gradually shows its advantages, and the classification precision and recall eventually surpass FineWP by about 4%. However, the improvement of classification accuracy of WFCNN is at the expense of training time. Table X shows the training time of the 5 methods in the open-world scenario. We can see that WFCNN takes nearly 30 times longer than other methods. Therefore, in the open-world scenario, FineWP can achieve near optimal precision and recall and has a much faster training process than WFCNN.

E. Evaluation on Robustness of FineWP

In this subsection, we are dedicated to investigating how potential influencing factors affect the performance of FineWP.

1) *Network Environment*: To verify whether FineWP is robust to different network conditions, we conduct experiments in which the testing data comes from an entirely

different network. As mentioned in Section V, the traces shown in Table IV are collected on 4 Ali cloud servers that are located in different regions in China. Thus, these servers have different network conditions.

We use the traces from one server as the testing data while the rest traces as the training data, and record the accuracy of the 4 rounds of tests covering all possible combinations of training and testing datasets, which are 91.1%, 91.6%, 92.0%, and 91.5% for YH dataset, and 81.1%, 82.0%, 81.2%, and 81.3% for JD dataset. For JD and YH traffic, no matter which dataset is used for testing, the accuracy keeps relatively stable. The average accuracy is 91.6% and 81.4% for YH and JD datasets, respectively, which are almost the same as the 10-fold cross-validation results shown in Fig. 11. The results confirm that FineWP is robust to network differences.

In the data collection process, we use cloud servers to simulate web fetching behaviors of individual users. As mentioned in Section VI-A, by using the server-level cross-validation, we can ensure that there is no overlap between users in the training and testing data to avoid learning on artifacts specific for individual users. The experimental results above confirm that FineWP can be generalized to different users.

2) *Time-Varying Webpage Changes*: To evaluate the effectiveness of FineWP in dealing with webpage content updates, we collect more traces of JD and YH for testing. For JD, we randomly select 55 webpages from the 330 monitored webpages in Table IV, each of which has 60 visits, and collect 3,300 webpage traces. For YH, we collect 800 traces from 20 randomly selected webpages that cover 10 categories.

In the following experiments, we use the dataset in Table IV to train FineWP in advance, and then use the newly collected traces for testing. The time difference between the training and testing data is over 10 months.

For JD, the classification accuracy is up to 80.2%, which is only slightly lower than the result (81.9%) in Fig. 11. The result confirms that the traffic generated from the same webpages on JD keep stable over time, and FineWP can accurately predict the webpage traces over a long-time span.

For YH, the classification accuracy is only 16.8%. It is because the contents of the webpages for testing have been changed greatly, thus making the values of the features used in FineWP change a lot. However, we use the 10-fold cross-validation to retrain FineWP with the newly collected traces and obtain an average accuracy of 90.0%, which approximates the result in Fig. 11. By comparing the updated FineWP with the previous one, we find the parameters they have in common: the number of uplink-only blocks (i.e., $K = 3$), the range of sequence features (between 25 and 83), and the 6 statistical features (i.e., *alen*, *ulen*, *ustd*, *dlen*, *umax* and *uvar*), as mentioned in Section VI-B. The only difference between them is the values of features, including block features, sequence features and statistical features. The reason lies in that the traffic pattern still follows the three-stage transmission rule and we only need to update the feature values of FineWP to achieve desirable accuracy. The frequency of classifier retraining will be discussed in Section VII.

3) *Webpage Fingerprinting Defenses*: Packet padding has been proposed as an effective defense against traffic

analysis [18], which makes all packet lengths become the same. To investigate how packet padding affects the effectiveness of FineWP, we leverage the JD and YH traces in Table IV and pad each packet with the maximum length of all packets (1514 Bytes). The accuracy of FineWP reduces from 81.9% to 61.8% on JD dataset and from 92.4% to 79.2% on YH dataset. The results above demonstrate that packet padding can render FineWP less effective. However, FineWP can still work, as the three-stage transmission pattern exists in the defended traffic.

VII. DISCUSSION

The observation in our experiments naturally raises a question: Why the CNNs-based classifier has poor performance on webpage fingerprinting? As we know that the maximum length of packets is determined by the Maximum Transmission Unit (MTU). In the process of client-server interaction, a number of packets will be fully loaded with the maximum packet length to transmit webpage contents, and many ACK packets with a fixed packet length will be sent to confirm whether the communication correspondent has received certain packets. Therefore, packet length sequence has a limited diversity and makes the CNNs-based classifier have relatively low accuracy.

We further discuss how MTU influences FineWP. Take the webpage fetching for example, a smaller MTU will lead to more packets in the generated traffic sequence, as the valid data in each packet is reduced. If MTU changes, FineWP still works, as the traffic generated by visiting webpages always follow the three-stage pattern as shown in Fig. 1. To achieve high accuracy, however, we should update the values of features used in FineWP, as mentioned in Section VI-B.

Next, we discuss the limitations of the proposed FineWP.

The first one is that webpage contents (e.g., pictures and texts) would change over time, leading to changes in the resulting traffic. Therefore, we need to retrain the classifier periodically. FineWP is built on the characteristics in client-server interaction. We have demonstrated in previous section that these characteristics are relatively stable over time but their specific values may change. In practice, one can keep on collecting test samples and use the trained classifier for prediction: if the testing accuracy falls below a user-specific threshold, it indicates the classifier needs to be retrained.

The second limitation is that the impact of TLS library on FineWP is not well evaluated in this paper. The underlying TLS library can influence the way information is structured across TCP segments, thus leading to pattern variations in the resulting traffic. Therefore, it may impact the performance of FineWP and more investigation is needed.

Third, FineWP has limited capabilities in dealing with traffic analysis defenses [10], [12], [18], such as packet padding and IP address masking. As demonstrated in Section VI-E, packet padding will degrade the performance of FineWP, and can further reduce its accuracy when dummy packets are inserted into the original sequences. IP address masking techniques (e.g., Tor) make it difficult to remove advertisement packets in traffic preprocessing, thus resulting in unstable traffic patterns when advertisements change frequently. To address this problem, we need to explore more distinguishable feature representations and powerful classifiers.

We will leave these attempts for designing more powerful attacks as the future work.

VIII. CONCLUSION

In this paper, we proposed an efficient fine-grained webpage fingerprinting named FineWP using only packet length information. Instead of just utilizing website homepages as the representative webpages, we analyzed multiple webpages within a given websites. Based on the traffic features of bidirectional interaction between clients and servers, we constructed the U0 sequence and extracted three types of features to build webpage fingerprints. Experimental results shown that the accuracy of FineWP achieved more than 20% improvement on JD dataset and more than 10% improvement on YH dataset over the state-of-the-art methods. Additionally, FineWP was 30× faster than the CNNs-based method in terms of training time. In the future work, we plan to further expand the dataset and consider applying deep learning to webpage fingerprinting in a more accurate and efficient way.

REFERENCES

- [1] Ali Cloud. Accessed: Oct. 1, 2018. [Online]. Available: <https://www.aliyun.com/product/ecs>
- [2] Jingdong. Accessed: Oct. 1, 2018. [Online]. Available: <https://www.jd.com>
- [3] SNI. Accessed: Oct. 1, 2018. [Online]. Available: https://en.wikipedia.org/wiki/Server_Name_Indication
- [4] Website Ranking. Accessed: Oct. 1, 2018. [Online]. Available: https://en.wikipedia.org/wiki/List_of_most_popular_websites
- [5] Yahoo!. Accessed: Oct. 1, 2018. [Online]. Available: <https://www.yahoo.com>
- [6] Y. Chen, R. Mahajan, B. Sridharan, and Z.-L. Zhang, "A provider-side view of Web search response time," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 43, no. 4, pp. 243–254, Sep. 2013.
- [7] T. Dierks and E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.2*, RFC document 5246, 2008. [Online]. Available: <https://tools.ietf.org/html/rfc5246>
- [8] Y.-N. Dong, J.-J. Zhao, and J. Jin, "Novel feature selection and classification of Internet video traffic based on a hierarchical scheme," *Comput. Netw.*, vol. 119, pp. 102–111, Jun. 2017.
- [9] R. Dubin, A. Dvir, O. Pele, and O. Hadar, "I know what you saw last minute—Encrypted HTTP adaptive video streaming title classification," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 12, pp. 3039–3049, Dec. 2017.
- [10] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-boo, i still see you: Why efficient traffic analysis countermeasures fail," in *Proc. IEEE Symp. Secur. Privacy*, May 2012, pp. 332–346.
- [11] S. Feghhi and D. J. Leith, "A Web traffic analysis attack using only timing information," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 8, pp. 1747–1759, Aug. 2016.
- [12] S. Feghhi and D. J. Leith, "An efficient Web traffic defence against timing-analysis attacks," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 2, pp. 525–540, Feb. 2019.
- [13] A. Freier, P. Karlton, and P. Kocher, *The Secure Sockets Layer (SSL) Protocol Version 3.0*, RFC document 6101, Aug. 2011.
- [14] Y. Fu *et al.*, "Service usage analysis in mobile messaging apps: A multi-label multi-view perspective," in *Proc. IEEE 16th Int. Conf. Data Mining (ICDM)*, Dec. 2016, pp. 877–882.
- [15] Y. Fu, H. Xiong, X. Lu, J. Yang, and C. Chen, "Service usage classification with encrypted Internet traffic in mobile messaging apps," *IEEE Trans. Mobile Comput.*, vol. 15, no. 11, pp. 2851–2864, Nov. 2016.
- [16] J. Gong and T. Wang, "Zero-delay lightweight defenses against Website fingerprinting," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 1–18.
- [17] E. Grolman *et al.*, "Transfer learning for user action identification in mobile apps via encrypted traffic analysis," *IEEE Intell. Syst.*, vol. 33, no. 2, pp. 40–53, Mar. 2018.
- [18] G. Kadianakis, "Packet size pluggable transport and traffic morphing," Tor Project, Seattle, WA, USA, Tech. Rep. 2012-03-004, 2012.

- [19] M. Korczynski and A. Duda, "Markov chain fingerprinting to classify encrypted traffic," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Apr. 2014, pp. 781–789.
- [20] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [21] A. Panchenko *et al.*, "Website fingerprinting at Internet scale," in *Proc. Neww. Distrib. Syst. Secur. Symp.*, 2016, pp. 1–15.
- [22] D. V. Patil and R. S. Bichkar, "A hybrid evolutionary approach to construct optimal decision trees with large data sets," in *Proc. IEEE Int. Conf. Ind. Technol.*, Dec. 2006, pp. 429–433.
- [23] G. Ranjan, A. Tongaonkar, and R. Torres, "Approximate matching of persistent lexicon using search-engines for classifying mobile app traffic," in *Proc. 35th Annu. IEEE Int. Conf. Comput. Commun.*, Apr. 2016, pp. 1–9.
- [24] S. Rezaei and X. Liu, "Deep learning for encrypted traffic classification: An overview," *IEEE Commun. Mag.*, vol. 57, no. 5, pp. 76–81, May 2019.
- [25] V. Rimmer, D. Preuveneers, M. Juarez, T. V. Goethem, and W. Joosen, "Automated website fingerprinting through deep learning," in *Proc. Neww. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA, 2018, pp. 1–15.
- [26] M. Shen, Y. Liu, S. Chen, L. Zhu, and Y. Zhang, "Webpage fingerprinting using only packet length information," in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2019, pp. 1–6.
- [27] M. Shen, Y. Liu, L. Zhu, K. Xu, X. Du, and N. Guizani, "Optimizing feature selection for efficient encrypted traffic classification: A systematic approach," *IEEE Netw.*, vol. 34, no. 4, pp. 20–27, Jul. 2020.
- [28] M. Shen, M. Wei, L. Zhu, and M. Wang, "Classification of encrypted traffic with second-order Markov chains and application attribute bigrams," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 8, pp. 1830–1843, Aug. 2017.
- [29] M. Shen, M. Wei, L. Zhu, M. Wang, and F. Li, "Certificate-aware encrypted traffic classification using second-order Markov chain," in *Proc. IEEE/ACM 24th Int. Symp. Qual. Service (IWQoS)*, Beijing, China, Jun. 2016, pp. 1–10.
- [30] M. Shen, J. Zhang, K. Xu, L. Zhu, J. Liu, and X. Du, "DeepQoE: Real-time measurement of video QoE from encrypted traffic with deep learning," in *Proc. IEEE/ACM 28th Int. Symp. Qual. Service (IWQoS)*, Hangzhou, China, Jun. 2020, pp. 1–10.
- [31] M. Shen, J. Zhang, L. Zhu, K. Xu, X. Du, and Y. Liu, "Encrypted traffic classification of decentralized applications on ethereum using feature fusion," in *Proc. Int. Symp. Qual. Service*, Phoenix, AZ, USA, Jun. 2019, p. 18.
- [32] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*. Accessed: Oct. 1, 2018. [Online]. Available: <https://arxiv.org/abs/1409.1556>
- [33] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. 2018 Conf. Comput. Commun. Secur. (CCS)*, 2018, pp. 1928–1943.
- [34] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Robust smartphone app identification via encrypted network traffic analysis," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 1, pp. 63–78, Jan. 2018.
- [35] Y. Udovychenko, A. Popov, and I. Chaikovsky, "Ischemic heart disease recognition by k-NN classification of current density distribution maps," in *Proc. IEEE 35th Int. Conf. Electron. Nanotechnol. (ELNANO)*, Apr. 2015, pp. 402–405.
- [36] T. Wang, X. Cai, R. Nithyanand, R. Johnson, and I. Goldberg, "Effective attacks and provable defenses for Website fingerprinting," in *Proc. 23rd Usenix Conf. Secur. Symp.*, 2014, pp. 143–157.



Meng Shen (Member, IEEE) received the B.Eng. degree in computer science from Shandong University, Jinan, China, in 2009, and the Ph.D. degree in computer science from Tsinghua University, Beijing, China, in 2014. He is currently with the Beijing Institute of Technology, Beijing as an Associate Professor. His research interests include privacy protection for cloud and IoT, blockchain applications, and encrypted traffic classification. He received the Best Paper Runner-Up Award at IEEE IPCCC 2014.



Yiting Liu received the B.Eng. degree in computer science from Northwest A&F University, Shanxi, China in 2017, and the master's degree in computer science from the Beijing Institute of Technology, Beijing, China, in 2020. She is currently pursuing the Ph.D. degree with the School of Computer Science, Fudan University, Shanghai, China. Her research interests include cybersecurity and artificial intelligence.



Liehuang Zhu (Member, IEEE) is currently a Professor with the Department of Computer Science, Beijing Institute of Technology. He is selected into the Program for New Century Excellent Talents in University from Ministry of Education, China. His research interests include Internet of Things, cloud computing security, and Internet and mobile security.



Xiaojiang Du (Fellow, IEEE) received the B.S. and M.S. degrees in electrical engineering from Tsinghua University, Beijing, China, in 1996 and 1998, respectively, and the M.S. and Ph.D. degrees in electrical engineering from the University of Maryland College Park in 2002 and 2003, respectively. He is currently a tenured Professor with the Department of Computer and Information Sciences, Temple University, Philadelphia, USA. He has been awarded more than U.S. \$5 million research grants from the U.S. National Science Foundation (NSF), Army Research Office, Air Force, NASA, the State of Pennsylvania, and Amazon. He has authored over 400 journal and conference papers in these areas, as well as a book published by Springer. His research interests are wireless communications, wireless networks, security, and systems. He received the Best Paper Award at IEEE GLOBECOM 2014 and the Best Poster Runner-Up Award at the ACM MobiHoc 2014. He serves on the editorial board of three international journals. He is a Life Member of ACM.



Jiankun Hu (Senior Member, IEEE) is currently a Professor with the School of Engineering and IT, University of New South Wales, Canberra, Australia. He is an invited expert of the Australia Attorney-General's Office assisting the draft of Australia National Identity Management Policy. He has many publications in top venues, including IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, *Pattern Recognition*, and IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS. His research interest is in the field of cybersecurity covering intrusion detection, sensor key management, and biometrics authentication. He has served at the Panel on Mathematics, Information and Computing Sciences, Australian Research Council ERA (The Excellence in Research for Australia) Evaluation Committee in 2012. He is an Associate Editor of IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY.